

Train One Sparse Autoencoder Across Multiple Sparsity Budgets to Preserve Interpretability and Accuracy

Anonymous ACL submission

Abstract

Sparse Autoencoders (SAEs) have proven to be powerful tools for interpreting neural networks by decomposing hidden representations into disentangled, interpretable features via sparsity constraints. However, conventional SAEs are constrained by the fixed sparsity level chosen during training; meeting different sparsity requirements therefore demands separate models and increases the computational footprint during both training and evaluation. We introduce a novel training objective, *HierarchicalTopK*, which trains a single SAE to optimise reconstructions across multiple sparsity levels simultaneously. Experiments with Gemma-2 2B demonstrate that our approach achieves Pareto-optimal trade-offs between sparsity and explained variance, outperforming traditional SAEs trained at individual sparsity levels. Further analysis shows that *HierarchicalTopK* preserves high interpretability scores even at higher sparsity. The proposed objective thus closes an important gap between flexibility and interpretability in SAE design.

1 Introduction

Transformers have revolutionised natural language processing (NLP) by achieving state-of-the-art performance across diverse tasks. Yet their internal representations remain notoriously difficult to interpret, often exhibiting *polysemanticity*, in which individual neurons activate for semantically unrelated features. To address this challenge, recent work has focused on Sparse Autoencoders (SAEs), which learn disentangled, human-interpretable directions in Transformer residual streams by enforcing sparsity constraints on the latent representations.

SAEs decompose hidden states into latent embeddings that are theoretically grounded in the independent additivity principle (Ayonrinde et al., 2024). This principle posits that individual features contribute to model behaviour independently, enabling isolated analysis of the latents. In practice,

relaxing sparsity constraints (e.g. increasing the number of active latents) often introduces entanglement: latents begin to co-activate for unrelated features, undermining interpretability. Consequently, the effectiveness of existing SAEs is tightly coupled to a single sparsity level fixed during training.

We propose *HierarchicalTopK*, a novel activation mechanism and training objective that enables a single SAE to maintain interpretable features across a range of sparsity levels. Unlike conventional SAEs, which must be retrained to accommodate different sparsity requirements, our method ensures that any subset of latents with $k \leq K$ remains disentangled and faithful to the independent additivity principle. Empirically, *HierarchicalTopK* SAEs achieve Pareto-optimal trade-offs between sparsity and explained variance, outperforming traditional SAEs trained independently at varying sparsity levels. This work bridges the gap between flexibility and interpretability in SAE design, enabling dynamic adaptation to downstream tasks with varying computational or fidelity requirements.

2 Method

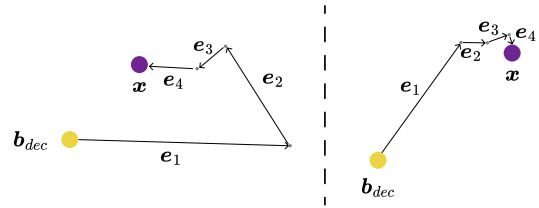


Figure 1: **Left:** SAE trained on a single k . **Right:** SAE trained on all $k \leq K$.

A sparse autoencoder (SAE) is defined as

$$l = \sigma(W_{\text{enc}}x + b_{\text{enc}}),$$

$$\hat{x} = W_{\text{dec}}l + b_{\text{dec}},$$

where $W_{\text{enc}} \in \mathbb{R}^{D \times h}$, $W_{\text{dec}} \in \mathbb{R}^{h \times D}$, $b_{\text{enc}} \in \mathbb{R}^D$, and $b_{\text{dec}} \in \mathbb{R}^h$. Here, D is the dictionary size and

h the hidden dimension. The non-linearity $\sigma(\cdot)$ is central. Vanilla SAEs use ReLU (Bricken et al., 2023), requiring an additional sparsity penalty on the latents. Sparsity can instead be induced directly with activations such as TOPK (Makhzani and Frey, 2013) or BATCHTOPK (Bussmann et al., 2024). Our analysis focuses on these activation variants.

The decoder can be viewed as a set of embeddings $W_{\text{dec}} = [e_1, \dots, e_D]$, yielding

$$\hat{x} = \sum_{i \in \text{top}_k} l_i(x) e_i + b_{\text{dec}},$$

where l_i is the i -th component of l . Embeddings are thus scaled by $l_i(x)$ to reconstruct x . The reconstruction error is $\mathcal{L}_{\text{rec}} = \|x - \hat{x}\|^2$. Optimising $\mathcal{L}_{\text{rec}}$ for a fixed top_k can be suboptimal when one wishes to interpret individual directions (small k).

Hierarchical loss. We therefore introduce a *hierarchical* loss. Define

$$\hat{x}_j = \sum_{i \in \text{top}_j} l_i(x) e_i + b_{\text{dec}},$$

$$\mathcal{L}_{\text{rec}}^j = \|x - \hat{x}_j\|^2,$$

for $j \in \mathcal{J} \subset \mathbb{N}$ (e.g. $\mathcal{J} = \{1, \dots, k\}$). The overall objective is

$$\mathcal{L}_{\text{hierarchical}} = \frac{1}{|\mathcal{J}|} \sum_{j \in \mathcal{J}} \mathcal{L}_{\text{rec}}^j. \quad (1)$$

Whereas the standard SAE guarantees reconstruction only at k active embeddings, our formulation encourages good reconstructions for every $j \leq k$. The optimal model under $\mathcal{L}_{\text{hierarchical}}$ therefore improves $\hat{x}_j$ monotonically with increasing j , a property absent in the vanilla SAE.

The hierarchical loss is inexpensive: it can be computed in a single forward pass via a cumulative-sum operation and implemented with kernels that avoid materialising intermediate tensors. In our implementation it runs faster than the original TopK loss; see Appendix C for details.

3 Experiments

3.1 Setup

For our experiments, we chose the Gemma-2 2B model (Gemma Team, 2024). We trained SAEs on a 1 B-token subsample of the FineWeb dataset (Penedo et al., 2024). Unless stated otherwise, we use the output of the 12th Transformer layer and set

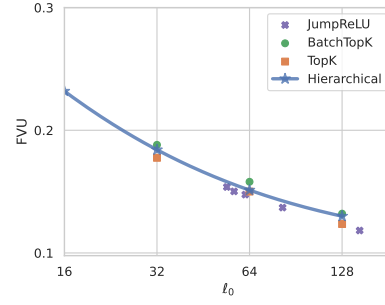


Figure 2: Comparison of an SAE with Hierarchical activation against other activation variants. The proposed method lies on the Pareto-optimal frontier across all sparsity levels, even though it is a single model.

the SAE dictionary size to $D = 65\,536$. Training details are provided in Appendix A.

We report the fraction of unexplained variance (FVU) as the main metric:

$$\text{FVU}(x, \hat{x}) = \frac{\text{Var}(x - \hat{x})}{\text{Var}(x)}.$$

Sparsity is measured by the ℓ_0 norm

$$\ell_0 = \sum_i \mathbf{I}[l_i > 0].$$

Because we use TopK-based activations, $\ell_0 = k$.

3.2 Hierarchical SAE Pareto Frontier

To evaluate the proposed training technique, we trained baseline SAEs at different sparsity levels. Specifically, we trained JumpReLU (Rajamanoharan et al., 2024) with various sparsity-regularisation coefficients and TopK and BatchTopK SAEs with $k \in \{32, 64, 128\}$. We also trained a single HierarchicalTopK SAE with $K = 128$. Figure 2 shows that our model matches or surpasses the performance of the individually trained baselines across all sparsity levels while requiring only one set of parameters.

3.3 Changing ℓ_0 at the Inference

To assess generalisation across sparsity levels, we trained a single HierarchicalTopK SAE with $K = 128$ and baseline TopK and BatchTopK SAEs with fixed $k \in \{32, 64, 128\}$. At inference we varied ℓ_0 over a dense grid, including both interpolation points within the training range and extrapolation points outside it. As shown in Figure 3, the Hierarchical model performs as well as—or better than—the baselines for $\ell_0 \leq 128$, demonstrating

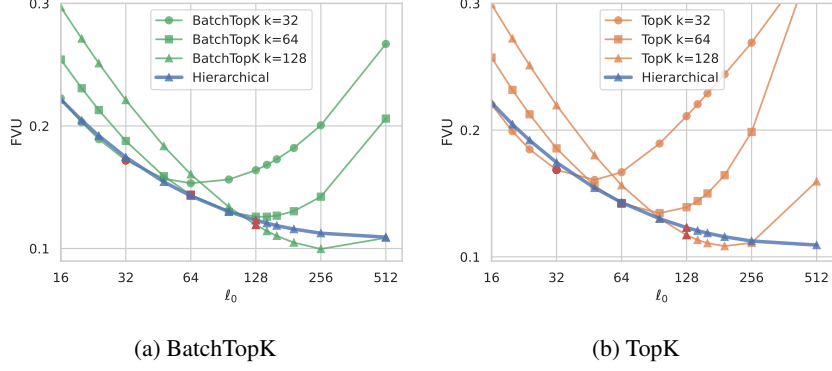


Figure 3: Pareto frontier for SAEs with BatchTopK, TopK, and Hierarchical activation functions. Red dots denote the ℓ_0 values on which the BatchTopK and TopK SAEs were trained. HierarchicalTopK matches or surpasses separately trained BatchTopK and TopK SAEs when interpolating ($\ell_0 \leq 128$), allowing a single SAE to select ℓ_0 post-training. See Section 3.3 for details.

that training across multiple k values is crucial for robust performance.

BatchTopK mixes different k values between samples during training, resulting in a primitive form of extrapolation. Consequently, it continues to improve reconstructions for $\ell_0 \in [128, 512]$, a sparsity range rarely used in practice.

3.4 Pointwise Loss

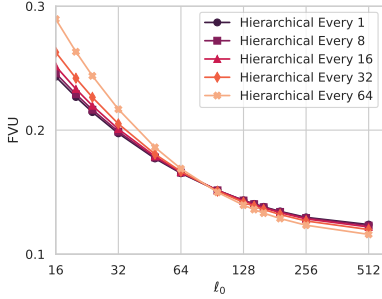


Figure 4: We test simple heuristics to reduce the computation required to train HierarchicalTopK. Computing the loss on every 8th term does not affect performance; see Section 3.4 for details.

To reduce computational overhead we evaluated computing the hierarchical loss on a subsampled index set (Equation 1):

$$J_x = \{1\} \cup \{i \in \mathbb{N} : i \bmod x = 0 \wedge 1 < i \leq k\},$$

with $x \in \{1, 8, 16, 32, 64\}$. As Figure 4 shows, computing the loss on every 8th term ($x = 8$) yields performance indistinguishable from the full loss, providing an eight-fold theoretical reduction in FLOPs.

An SAE whose loss is calculated on every 64th term suffers a significant performance decrease for

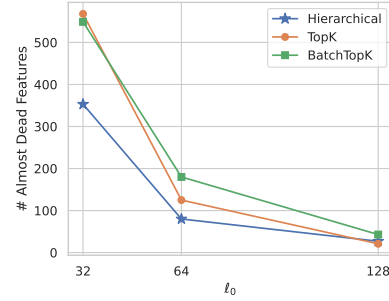


Figure 5: Number of features with activation frequency below 10^{-5} (“almost dead”) for SAEs trained with $k = 128$. “Optimal scaling” denotes the number of almost-dead features in a BatchTopK SAE trained with $k = \ell_0$. The BatchTopK model accumulates almost-dead features more rapidly than the Hierarchical model when k is reduced at inference time; see Section 3.5 for details.

$\ell_0 < 128$, but extrapolates better for $\ell_0 > 128$. Remarkably, using the hierarchical loss on every 8th term ($J_8 = \{1, 8, 16, 24, 32, \dots, 128\}$) reduces theoretical overhead by a factor of eight without sacrificing reconstruction quality. In practice, however, there is almost no difference in per-step training time between vanilla TopK and HierarchicalTopK; see Appendix C for details.

3.5 Why SAE Struggle to Reduce ℓ_0 ?

To investigate why simple SAE variants struggle to interpolate to lower ℓ_0 values than those used during training, we measured the number of features whose activation frequency falls below 10^{-5} (i.e. they activate once in 10^5 tokens). We call these features *almost dead*.

We trained TopK and BatchTopK models with

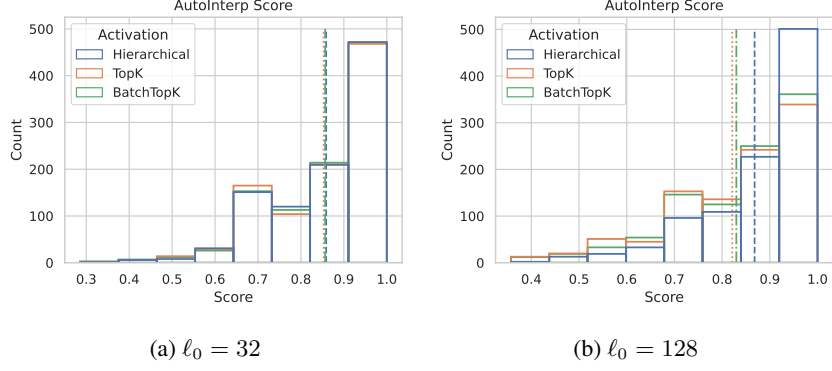


Figure 6: AutoInterp Score (Paulo et al., 2024). TopK and BatchTopK scores are obtained from two separate SAEs trained with $k = 32$ and $k = 128$; the Hierarchical model uses a single SAE trained on all $k \leq 128$. Hierarchical activation preserves the interpretability level of SAEs trained with smaller ℓ_0 .

$k = 128$ and then, following Section 3.3, evaluated them with $k \in \{32, 64, 128\}$. The Hierarchical variant was trained once with $k = 128$. Results are shown in Figure 5. Although all SAEs exhibit similar numbers of dead features at $\ell_0 = 128$, the Hierarchical model keeps significantly more features alive than the TopK and BatchTopK variants as k decreases.

3.6 Interpretability

To validate interpretability we use the detection score of Paulo et al. (2024), implemented in SAE Bench (Karvonen et al., 2025). For TopK and BatchTopK we evaluate two SAEs trained with $k = 32$ and $k = 128$; for Hierarchical we evaluate a single SAE trained on all $k \leq 128$ (see Figure 6).

For the Hierarchical SAE the interpretability score at $\ell_0 = 128$ is almost identical to that at $\ell_0 = 32$, while its explained variance remains on the Pareto frontier (Figure 2). By contrast, in both TopK and BatchTopK variants, less-sparse models tend to be less interpretable. This observation underscores the superiority of the hierarchical loss.

4 Related Work

Research on sparse autoencoders increasingly focuses on feature interpretability in Transformer representations. The seminal work of Gao et al. (2025) introduced TopK-sparse autoencoders; Bussmann et al. (2024) extended this idea with batch-level sparsity control. However, these approaches lack a mechanism for establishing feature importance or relationships.

Structural constraints have also been explored. Bussmann et al. (2025) and Ayonrinde et al. (2024) investigate hierarchical dictionaries, demonstrating

the benefits of progressive refinement. Building on these insights, our training method naturally encodes feature importance through progressive reconstruction, mirroring gradient-descent dynamics and feature hierarchies while maintaining interpretability and improving generalisation across sparsity levels.

5 Conclusion

We introduced *HierarchicalTopK*, a single sparse-autoencoder objective that enforces high-quality reconstructions at every sparsity level up to a chosen budget K . Experiments on Gemma-2 2B representations show that our approach:

- Achieves Pareto-optimal trade-offs between explained variance and ℓ_0 compared with independently trained TopK and BatchTopK baselines, despite using a single model.
- Maintains high interpretability across sparsity levels and prevents the proliferation of “dead” features when ℓ_0 is varied at inference time.

These contributions provide a flexible, efficient, and interpretable framework for analysing Transformer latent spaces under varying computational constraints.

6 Limitations

Our work has two principal limitations. (i) **Evaluation scope:** experiments are limited to the Gemma-2 2B model and a FineWeb subset; transfer to other architectures and datasets remains to be tested. (ii) **Interpretability measures:** we rely on automated metrics as proxies for human judgement; user studies are needed to validate semantic alignment.

References

- Kola Ayonrinde, Michael T. Pearce, and Lee Sharkey. 2024. [Interpretability as compression: Reconsidering sae explanations of neural activations with mdl-saes](#). *Preprint*, arXiv:2410.11179.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, and 6 others. 2023. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- Bart Bussmann, Patrick Leask, and Neel Nanda. 2024. Batchtopk sparse autoencoders. *arXiv preprint arXiv: 2412.06410*.
- Bart Bussmann, Noa Nabeshima, Adam Karvonen, and Neel Nanda. 2025. [Learning multi-level features with matryoshka sparse autoencoders](#). *Preprint*, arXiv:2503.17547.
- Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. 2025. [Scaling and evaluating sparse autoencoders](#). In *The Thirteenth International Conference on Learning Representations*.
- Google DeepMind Gemma Team. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv: 2408.00118*.
- Adam Karvonen, Can Rager, Johnny Lin, Curt Tigges, Joseph Bloom, David Chazin, Yeu-Tong Lau, Eoin Farrell, Callum McDougall, Kola Ayonrinde, Matthew Wearden, Arthur Conmy, Samuel Marks, and Neel Nanda. 2025. Saebench: A comprehensive benchmark for sparse autoencoders in language model interpretability. *arXiv preprint arXiv: 2503.09532*.
- Alireza Makhzani and Brendan J. Frey. 2013. k-sparse autoencoders. *International Conference on Learning Representations*.
- OpenAI. 2021. Introducing triton: Open-source gpu programming for neural networks. <https://openai.com/index/triton/>. Accessed: 2021-07-24.
- Gonçalo Paulo, Alex Mallen, Caden Juang, and Nora Belrose. 2024. Automatically interpreting millions of features in large language models. *arXiv preprint arXiv: 2410.13928*.
- Guilherme Penedo, Hynek Kydlicek, Loubna Ben al-lal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. 2024. [The fineweb datasets: Decanting the web for the finest text data at scale](#). In *The Thirty-eight Conference on*

Neural Information Processing Systems Datasets and Benchmarks Track.

- Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. 2024. [Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders](#). *Preprint*, arXiv:2407.14435.

A SAE Training Details

All SAEs were trained with a modified version of the code from [Bussmann et al. \(2024\)](#). Hyperparameters are listed in Table 1. We use NVIDIA H100 80GB GPU and spent about 20 GPU-days of compute, including preliminary experiments.

Parameter	Value
Optimizer	Adam
Learning Rate	0.0008
# tokens	10^9
Dataset	FineWeb
Batch Size	8096
Decoder Normalization	True

Table 1: Hyperparameters used to train the SAEs. See Section A for more details.

We also used modified kernels from [Gao et al. \(2025\)](#); see Appendix C for details.

B Additional Results

B.1 Latent Structure

To support Figure 1 we measured the cosine similarity between feature embeddings in the reconstruction sum

$$\hat{x} = \sum_{i \in \text{top}_k} l_i(x) e_i + b_{\text{dec}},$$

In Figure 7, e_1 denotes the top-1 activation, and so on. Ideally, similarity should decrease monotonically as activation values diminish.

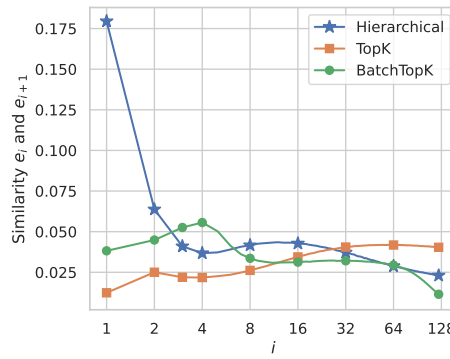


Figure 7: Cosine similarity of feature embeddings in the reconstruction sum.

Vanilla TopK SAEs show the undesired trend that similarity increases with the index i , whereas the Hierarchical model preserves the expected monotonic decrease.

B.2 Distribution of the Latents Activations

To compare the distributions learned by standard SAEs and the Hierarchical variant we analyse both feature frequency and mean-squared activation (Figure 8). Hierarchical training yields more latents with higher activation values (panel 8b), which may explain its superior interpretability. Its frequency distribution is skewed towards lower values, indicating that the hierarchical loss encourages activations to appear as the top-1 feature, enabling accurate reconstruction even at $k = 1$.

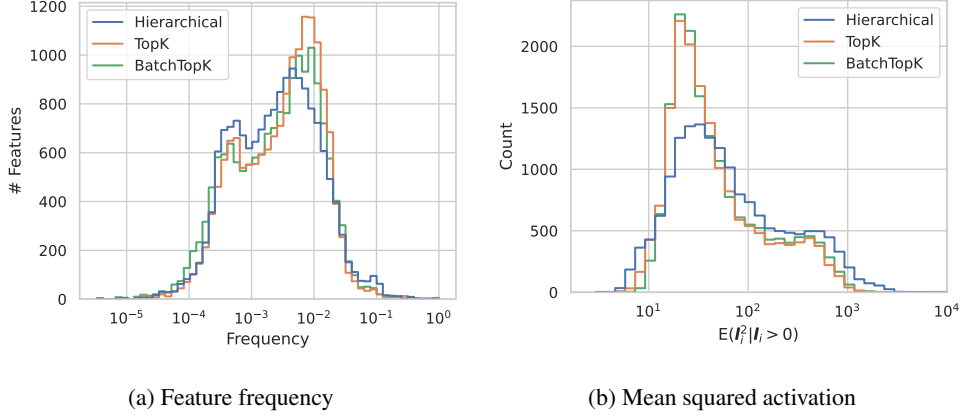


Figure 8: Latent-feature distributions for SAEs trained with $k = 128$ ($J = \{1, \dots, k\}$ in Hierarchical training).

B.3 JumpReLU and TopK evaluations

A BatchTopK SAE is trained with batch-wise sparsity but, if evaluated directly with per-token TopK, the training and inference settings mismatch. We therefore apply a constant-threshold JumpReLU at inference time, choosing the threshold so that the expected number of active features equals k . To study the effect of switching activations we trained SAEs with $\ell_0 = 64$ (the Hierarchical model was trained on $k \leq 128$) and evaluated every model at $\ell_0 = 64$. Results are shown in Figure 9.

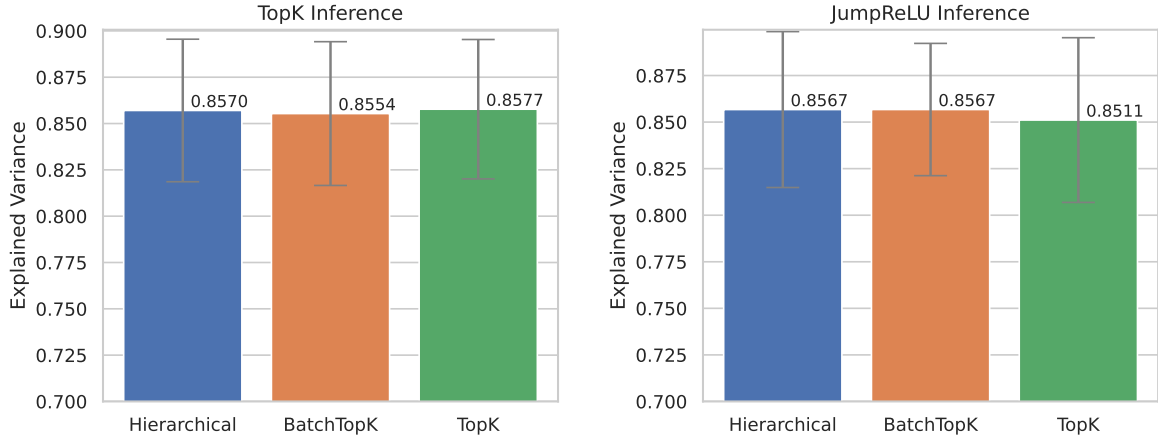


Figure 9: TopK versus JumpReLU inference. We did not find a significant difference between JumpReLU and fixed TopK evaluation.

The largest change in explained variance occurs for the TopK SAE, which drops from 0.8577 to 0.8511 under JumpReLU. BatchTopK improves marginally (+0.0013), and the Hierarchical variant is virtually unchanged for either activation.

C Implementation

We extend the Triton implementation (OpenAI, 2021)¹ of TopK-sparse autoencoders by fusing mean-squared-error computation directly into the sparse-decoder kernel. This produces both our optimised *TopKSAE* (with a fused loss) and *FlexSAE*. As shown in Table 2, both kernels surpass the baseline Triton implementation in speed while using the same peak memory.

In addition, we provide a minimal PyTorch implementation of the FlexSAE loss for clarity. This naïve version illustrates the core idea: we gather the decoder embeddings at the active indices, scale them by the

¹https://github.com/openai/sparse_autoencoder

SAE Method	Time per step (ms)	Peak Memory (MB)
Topk (Gao et al., 2025)	10.4800 ± 0.0836	6369.86
Fused Topk (Ours)	9.9235 ± 0.0368 (−5.31%)	6371.06
Fused Hierarchical (Ours)	10.0531 ± 0.0420 (−4.07%)	6371.06

Table 2: Training speed and memory usage for sparse-autoencoder kernels with batch size $B = 64$, model dimension $h = 2304$, dictionary size $D = 2^{16}$, and sparsity $\ell_0 = 128$.

sparse activations, compute a cumulative reconstruction, and measure the mean-squared error across all sparsity levels.

Listing 1: Naïve PyTorch implementation of the Hierarchical loss.

```

1  def hierarchical_loss(sparse_idx, sparse_val, decoder, b_dec, target):
2      """
3      sparse_idx: LongTensor of shape (B, K) with indices of active embeddings
4      sparse_val: FloatTensor of shape (B, K) with corresponding activation values
5      decoder:    FloatTensor of shape (D, h) containing the dictionary embeddings
6      b_dec:      FloatTensor of shape (h) containing decoder bias
7      target:     FloatTensor of shape (B, h) with the original inputs
8      """
9      B, K = sparse_idx.shape
10     flatten_idx = sparse_idx.view(-1)
11     emb = decoder[flatten_idx].view(B, K, -1)
12     emb = emb * sparse_val.unsqueeze(-1)
13
14     recon_cum = emb.cumsum(dim=1) + b_dec.unsqueeze(1)
15
16     diff = recon_cum - target.unsqueeze(1)
17     total_err = diff.pow(2).mean()
18     return total_err

```